Who will be back within 30 days?

Predicting early readmission of diabetic inpatients to plan hospital beds

Muhammad TALHA Jules BORDAIS
Elimane DIALLO

Supervised Learning & Model Engineering — Fall 2026

Dataset: Diabetes 130-US hospitals 1999–2008 (OpenML id 4541), 101,766 encounters, 71,518 patients.
Repository: `https://github.com/Elimane29/supervised_learning_project_Muhammad_Jules_Elimane`
Entry points: `scripts/01_eda.py` ... `09_leakage_checks.py`, `api/app.py`.

1 Problem framing

A hospital whose beds are under pressure wants to know, *at discharge*, which diabetic inpatients will be readmitted within 30 days, so that it can plan beds and target follow-up calls. The prediction unit is one encounter; the prediction is made at discharge, so every feature must pass the *prediction-time test*: could its value be known at that moment?

The original target has three classes (`NO`, `>30`, `<30`). We define $Y = 1$ for `<30` and $Y = 0$ otherwise: a return after 30 days does not strain this month's beds. The model estimates $p(x) = P(Y = 1 \mid x)$; it produces a risk, not a decision. Two downstream uses follow, and they shape our evaluation:
1. **Bed planning** uses the *sum* of probabilities, so probabilities must be calibrated (Section 8).
2. **Follow-up** uses a threshold on $p(x)$, chosen from the relative cost of the two errors (Section 7).
With a positive rate of 11.4%, a model that always predicts "no" reaches 88.6% accuracy and is useless. Our selection metric is therefore the **average precision** (AP, the step-wise PR-AUC), whose random baseline equals the prevalence (0.114). We also report ROC-AUC (ranking), the Brier score and the expected calibration error (ECE) (probability quality), and, at the decision threshold, accuracy, precision, recall, F1 and the expected cost. Accuracy is shown for completeness, not for selection: on this data it rewards the model that never flags anyone.

2 Data understanding and leakage audit

Cohort. Starting from 101,766 encounters (71,518 patients), we remove 2,423 encounters ending in death or hospice and 3 encounters with gender `Unknown/Invalid`; the two groups do not overlap. The modelling cohort therefore has $101{,}766 - 2{,}423 - 3 = 99{,}340$ encounters from 69,987 patients.

The EDA (`scripts/01_eda.py`, on the development split only) produced seven findings, each turned into a rule (Table 1).

Finding	Risk	Decision
2,423 encounters end in death or hospice (discharge ids 11, 13, 14, 19, 20, 21)	target leakage	Removed: these patients cannot be readmitted; keeping them teaches a trivially true, useless rule.
3 encounters with gender `Unknown/Invalid`	data quality	Removed: too few to learn from, and they would form an empty group in the fairness audit.
23% of patients have several encounters (up to 40)	group leakage	`StratifiedGroupKFold` on `patient_nbr` for the test split and every CV fold.
Earlier stays of a patient are informative	temporal leakage	History features count only encounters with a smaller `encounter_id`, never later ones.
`weight` 97% missing; `examide`, `citoglipton` constant	noise	Dropped. "`?`" kept as an explicit `Unknown` category: missingness may carry signal (e.g. no payer recorded).
Admission/discharge ids and age brackets stored as codes	semantics	Ids treated as categories; age bracket mapped to its midpoint (ordinal kept).
`race` is a sensitive attribute	fairness	Never a model input; kept only for the audit (Section 11).

Table 1: Data audit: from EDA findings to modelling rules.

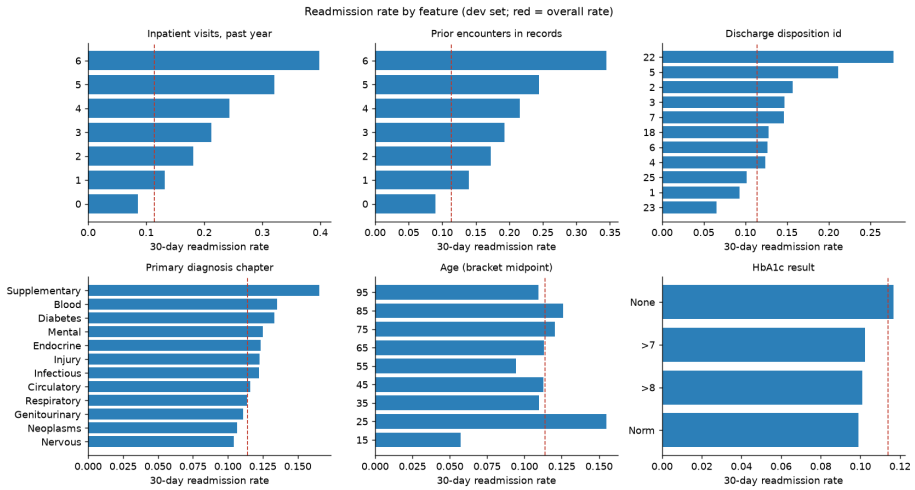


Figure 1: 30-day readmission rate by feature on the development split (red: overall 11.4%). Prior hospital use is the strongest univariate signal: from 8.5% with no inpatient visit in the past year to 40% with six. Patients without an HbA1c measurement are readmitted more often.

The numeric variables overlap heavily between the two classes (mean length of stay 4.3 vs 4.8 days), except prior utilisation: mean inpatient visits 0.56 vs 1.23 and mean prior encounters 0.55 vs 1.24. This already suggests a hard problem: much of what drives a readmission (adherence, social support) happens after discharge and is not recorded.

Measuring the leaks. To show what the audit protects against, `scripts/09_leakage_checks.py` rebuilds the leaky variants on purpose and evaluates them with the same LightGBM and the same metrics on the development set (Table 2).

Check	Variant	CV-val. AP ± sd	CV-val. ROC-AUC
Temporal	history = earlier stays only (ours)	0.238 ± 0.012	0.685
Temporal	history = earlier + later stays (leaky)	0.315 ± 0.010	0.789
Temporal	history = later stays only (leaky)	0.337 ± 0.008	0.800
Group	folds grouped by patient (ours)	0.238 ± 0.012	0.685
Group	patients mixed across folds (leaky)	0.240 ± 0.005	0.686

Table 2: Leaky variants rebuilt on purpose: CV-validation scores, mean ± sd over the 5 development folds. A higher score for a leaky variant is not a better model: the future is unknown at discharge.

The temporal leak is large. Counting a patient's later stays would raise AP from 0.238 to 0.315, and to 0.337 when only later stays are counted. That is a 32–42% inflation, and it would vanish in production, because a later stay is often the readmission itself.

The group leak is small here: 0.240 vs 0.238, within one fold standard deviation, although patients were shared between the CV-train and CV-validation parts 27,052 times. The model has no patient identifier and only weak patient-specific signals, so it cannot memorise individuals. We keep the grouped split anyway: it is the correct design for new patients, and it costs nothing.

3 Validation protocol

The cohort is split once with `StratifiedGroupKFold` (no patient shared, prevalence 11.39% on both sides). Throughout the report the sets are named as follows:

- **Development set** (80%, 79,473 encounters): everything used to build the model. It is cut into 5 patient-grouped, stratified folds, materialised once and reused by *every* model and Optuna trial, so all comparisons are paired.
- **CV-train / CV-validation**: in each of the 5 cross-validation rounds, the Pipeline is fitted on 4 folds (the *CV-train* part, about 63,600 encounters) and scored on the 5th (the *CV-validation* part, about 15,900). A *CV-validation score* is the mean ± standard deviation over the 5 rounds; the *out-of-fold predictions* are the 5 CV-validation predictions put together, one per development encounter, never made by a model that saw that encounter.
- **Training of the final model**: the frozen Pipeline is refitted once on the whole development set.
- **Test set** (20%, 19,867 encounters): held out and scored **once**, after every choice was frozen.

CV-validation scores and out-of-fold predictions are the single source for every decision: model comparison, hyperparameters, calibrator, decision threshold and fairness thresholds. Every score in this report is labelled with the phase it comes from: *CV-validation* (development) or *test*. `TimeSeriesSplit` was rejected: the data have no admission date, and encounter order alone does not define a deployment time. Nested CV was considered; with 80k development rows, CV tuning plus one untouched test set gives an honest estimate at a fraction of the cost (five times fewer fits).

4 Feature engineering and pipeline

All features are built by a scikit-learn transformer, `ReadmissionFeatures`, so that its only learned rule (which categories are too rare, < 50 occurrences, and are pooled into `Other`) is learned on the training part of each fold. Engineered features:

- `n_prior_encounters`: earlier stays of the patient in the records (past only, see Table 1); `total_prior_visits`: outpatient + emergency + inpatient visits in the past year.
- ICD-9 chapters for `diag_1/2/3`: about 700 codes mapped to 19 clinical groups (CDC/CMS ranges, with 250.xx split out as *Diabetes*); `diabetes_primary` flags a primary diabetic diagnosis.
- Treatment complexity: `n_meds_active` (diabetes drugs taken) and `n_meds_changed` (dose up or down); care intensity: `meds_per_day`, `labs_per_day`.
- `a1c_tested`, `glucose_tested`: whether the test was done at all ("None" means not measured).

Missing values, without leakage. The data contain no NaN: missing entries appear as the string "`?`" in five categorical columns. Mapping "`?`" to an explicit `Unknown` category is a fixed relabelling that learns nothing from the data, so it cannot leak whenever it is applied. The only learned imputation, the numeric median, sits inside the Pipeline. It is therefore fitted on the CV-train part of each fold, and on the full development set for the final model, never on CV-validation or test rows. It is a safeguard for API inputs, since no numeric value is missing in the dataset. A unit test (`test_imputer_learns_from_training_rows_only`) checks that the median comes from the training rows only and is not refitted on new rows.

The **Pipeline** (`src/readmission/pipelines.py`) chains `ReadmissionFeatures` → a `ColumnTransformer` → the model. Numeric features: median imputation (plus standardisation for the regularised logistic regression). Low-cardinality categoricals: one-hot encoding. The raw ICD-9 codes (high cardinality) are additionally target-encoded with scikit-learn's `TargetEncoder`, which *cross-fits* during `fit_transform`: a training row is never encoded with its own label, and rare codes are shrunk towards the mean. CatBoost receives the raw categories instead and encodes them itself with ordered target statistics. We use an `imblearn` Pipeline so that a resampler, when tested, acts only at fit time on the training fold. The deployed object is the fitted Pipeline plus calibrator plus threshold, loaded by the API.

LightGBM	CV-val. AP	CV-val. ROC-AUC
Full feature set	0.238	0.685
− prior encounters	0.237	0.681
− target encoding of ICD-9 codes	0.238	0.682
− both	0.236	0.677

Table 3: Feature ablation. The two additions give +0.009 ROC-AUC together; on AP the gain is smaller than the fold standard deviation (±0.012). We keep them: cheap, clinically meaningful and available at discharge.

5 Class imbalance

We compared remedies at three stages (Session 6) with LightGBM on the same folds; all resampling ran inside the training folds only.

Remedy (LightGBM)	CV-val. AP	CV-val. ROC-AUC	CV-val. Brier
None	0.238	0.685	**0.095**
Class weight $\sqrt{n_0/n_1} = 2.8$	**0.239**	0.686	0.113
Class weight $n_0/n_1 = 7.8$	0.235	0.683	0.202
Random undersampling to 1:2	0.227	0.679	0.140
Random oversampling to 1:2	0.235	0.683	0.132
Logistic regression, none → `balanced`	0.218 → 0.218	0.668 → 0.670	0.097 → 0.224

Table 4: Imbalance remedies: CV-validation scores on the development set; resampling applied to the CV-train part only.

No remedy improved the ranking beyond fold noise, and all of them inflated the predicted probabilities (Brier up to 0.20): re-weighting changes the training objective so that $p(x)$ no longer estimates the true frequency. Since our users read the probabilities, we chose to (i) measure with AP rather than accuracy, (ii) leave the class weight to Optuna as a hyperparameter rather than impose it, and (iii) handle the imbalance at the *decision* level with a cost-based threshold on calibrated probabilities, the least invasive remedy. SMOTE was not used: most inputs are categorical, and interpolating between diagnosis codes has no meaning.

6 Models, tuning and experiment tracking

We trained four model families of increasing flexibility, all inside the same Pipeline and evaluated on the same folds with the same metrics: a regularised logistic regression (interpretable linear baseline), a Random Forest (bagging of decorrelated trees), LightGBM and CatBoost (gradient boosting).

Optuna. Each family was tuned with Optuna (`scripts/03_tune.py`): the objective is the mean CV-validation AP over the 5 patient-grouped development folds, the full Pipeline is refitted in every fold, and the test set is never loaded. The TPE sampler proposes configurations from past trials; a median pruner stops a trial after a fold whose running mean is below the median of earlier trials. Search spaces covered regularisation (C, L1 vs L2), tree depth and leaves, learning rate jointly with the number of trees, row and column subsampling, and class weights. Elastic Net was excluded after measuring that the required `saga` solver was six times slower for the same AP. Budgets: LightGBM 32 trials, logistic 25, Random Forest 15, CatBoost 2 (each CatBoost fit takes about 2.5 minutes per fold; the 40-minute budget allowed two trials).

MLflow. Every run is logged in an MLflow tracking store (`mlflow.db`): one run per untuned experiment (17), one parent run per Optuna study with one nested child run per trial (parameters, per-fold AP and ROC-AUC, mean and standard deviation), and a final run holding the test metrics, figures, tables and the frozen model bundle. The 123 runs are exported to `reports/tables/mlflow_runs.csv`.

Model	CV-val. AP untuned	CV-val. AP tuned ± sd	CV-val. ROC-AUC	CV-val. Brier	Gain
Dummy (prevalence)	0.114	—	0.500	0.101	floor
Logistic regression (L1, $C = 0.21$)	0.218	0.219 ± 0.012	0.670	0.097	linear signal, ×1.9 the floor
Random Forest	0.227	0.227 ± 0.014	0.681	0.096	+0.008: thresholds, interactions
LightGBM	0.238	0.239 ± 0.013	0.686	0.097	+0.012: sequential corrections
CatBoost	0.248	**0.248 ± 0.015**	**0.688**	**0.095**	+0.009: ordered categorical encoding

Table 5: Model progression: CV-validation scores on the development set (mean over the 5 folds; tuned models re-evaluated on the same folds).

Each step of the progression bought about +0.01 AP; tuning bought almost nothing ($\leq$0.001), and when the class weight was a free hyperparameter Optuna chose none (logistic, Random Forest, CatBoost) or 1.4 (LightGBM), consistent with Section 5. The largest lever was how the high-cardinality categoricals (ICD-9 codes, specialty, discharge disposition) are represented, not the hyperparameters. CatBoost and LightGBM differ by less than one fold standard deviation; CatBoost also has the best Brier score, so it is the final model (iterations 900, depth 7, learning rate 0.067, $\ell_2 = 1.06$, no class weight). The best-of-N CV-validation score is slightly optimistic by construction, which is why the number we report for the final model comes from the test set.

				$t = 0.5$		$t = 0.175$ (deployed)			
Phase	Model	ROC-AUC	AP	Accuracy	F1	Accuracy	Precision	Recall	F1
CV-validation	Dummy (prevalence)	0.500	0.114	0.886	0.000	0.886	0.000	0.000	0.000
CV-validation	Logistic regression	0.670	0.218	0.886	0.033	0.828	0.255	0.265	0.260
CV-validation	Random Forest	0.680	0.225	0.886	0.000	0.816	0.248	0.303	0.273
CV-validation	LightGBM	0.686	0.238	0.886	0.041	0.721	0.205	0.507	0.292
CV-validation	**CatBoost**	**0.688**	**0.247**	0.887	0.056	0.814	0.258	0.336	0.292
Test	**CatBoost (final)**	0.683	0.239	0.887	0.046	0.809	0.249	0.336	0.286

Table 6: All metrics for all models. CV-validation rows use the pooled out-of-fold predictions of the development set (so AP differs slightly from the fold means of Table 5); the test row is the final model scored once. Source: `scripts/10_metrics_table.py`.

Table 6 shows why accuracy cannot select a model here. At the default threshold 0.5, every model reaches 0.886–0.887 accuracy, exactly the share of non-readmitted patients, because almost nobody is flagged (F1 $\leq$ 0.06). At the deployed threshold, the final model catches a third of readmissions (F1 0.29) while its accuracy *drops* to 0.81, below the model that never flags anyone. Precision and

recall make the trade-off visible. At the same threshold, LightGBM flags 28% of patients (recall 0.51, precision 0.21) against 15% for CatBoost (recall 0.34, precision 0.26): LightGBM's class weight of 1.4 inflates its probabilities, so one threshold does not mean the same operating point for both. Their F1 is identical (0.29), and CatBoost's calibrated probabilities make its threshold interpretable.

7 Test-set evaluation and decision threshold

Threshold. With calibrated probabilities, flagging a patient is worth it when $p \cdot C_{FN} > (1-p) \cdot C_{FP}$, i.e. $p > C_{FP}/(C_{FP} + C_{FN})$. We assume a missed early readmission (unplanned bed, emergency readmission) costs five times a false alert (one follow-up call or a provisional bed hold): $C_{FN} = 5$, $C_{FP} = 1$, so the theoretical threshold is $1/6 \approx 0.167$. A grid search on the out-of-fold (CV-validation) predictions of the development set gives 0.175, which we freeze. The cost curve is flat between 0.15 and 0.20 (expected cost 0.488–0.495 per discharge), so the choice is robust; with $C_{FN}/C_{FP} = 3$ or 10 the rule would move to 0.25 or 0.09.

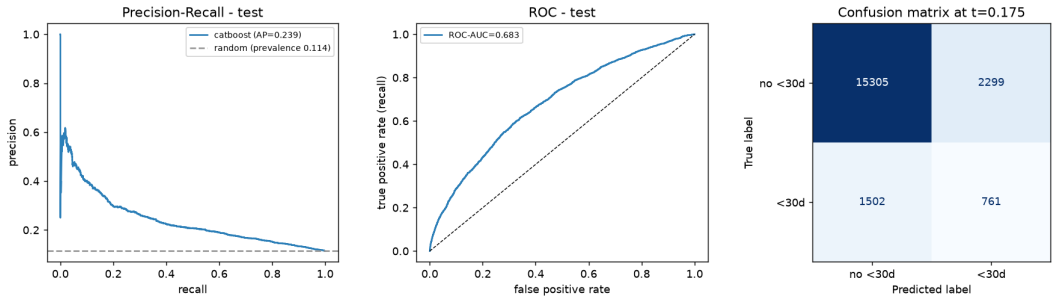


Figure 2: Final CatBoost on the untouched test set: precision–recall curve (AP 0.239 vs 0.114 at random), ROC curve (AUC 0.683) and confusion matrix at $t = 0.175$.

Metric	CV-validation (development)	**Test (once)**
Average precision	0.247	0.239
ROC-AUC	0.688	0.683
Brier / ECE	0.095 / 0.003	0.095 / 0.006
Patients flagged at $t = 0.175$	14.8%	15.4%
Recall / precision at t	0.336 / 0.258	0.336 / 0.249
Accuracy / F1 at t	0.814 / 0.292	0.809 / 0.286
TP / FP / FN / TN	—	761 / 2,299 / 1,502 / 15,305
Expected cost per discharge	0.488	0.494
baselines: flag nobody / flag all	0.570 / 0.886	0.570 / 0.886
Top 20% by risk: recall / precision	0.408 / 0.232	0.397 / 0.226

Table 7: Final model. The CV-validation and test numbers agree, which supports the protocol: no sign of optimistic selection.

The model catches one early readmission in three while flagging 15% of discharges, with alerts 2.2 times more precise than random, and reduces the expected cost by 13% relative to doing nothing. If follow-up capacity is the binding constraint (e.g. 20% of discharges), ranking by risk catches 40% of early readmissions. This is a prioritisation tool, not a diagnostic one: published results on this dataset sit in the same ROC-AUC range (0.65–0.70).

8 Calibration

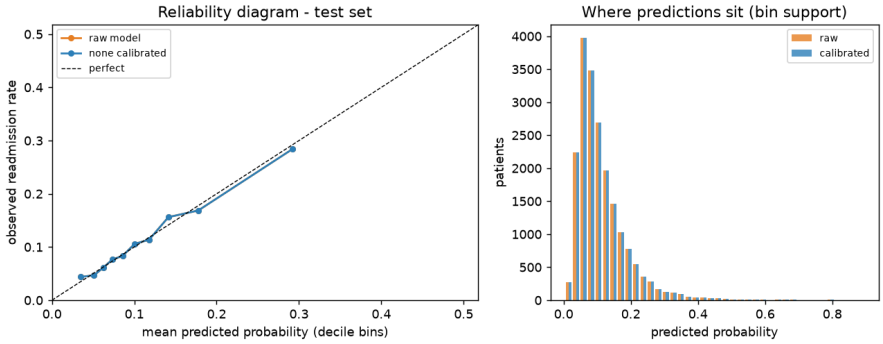


Figure 3: Reliability diagram on the test set (quantile bins, so that each point holds 10% of patients) and distribution of predictions. 89% of predictions are below 0.2, so uniform bins would leave the upper range almost empty.

The reliability curve follows the diagonal (Figure 3): test Brier 0.095 (0.101 for the base-rate predictor) and ECE 0.006. The model is **neither over- nor under-confident** overall; the top decile is marginally high (mean prediction 0.29 vs 0.28 observed), within noise for 1,987 patients. At the population level, the model expected **2,258 readmissions in the test set; 2,263 occurred**.

We compared three options without leakage: for each fold, a calibrator was fitted on the out-of-fold predictions of the other four folds and applied to the CV-validation fold. Platt scaling (on the logit) and isotonic regression did not lower the Brier score (0.0949–0.0950 for all three); isotonic regression lowered the ECE (0.001) but cost AP (0.242 vs 0.247) because its step function ties many scores. We therefore keep the model's probabilities unchanged. The reason it works is the unweighted log-loss training: with class weight 7.8 the Brier score of the same family rises to 0.20 (Table 4).

Would we trust these probabilities for a downstream decision as-is? Yes, for bed planning (the sum of probabilities) and for the cost-based threshold, which assumes calibration, on a population like this one. Before deployment we would re-check calibration per hospital and over time, since prevalence and practices drift (the data are from 1999–2008).

9 Diagnostics: where does the model fail?

A good global score can hide failures in important groups (Session 8). We therefore audited the frozen model at the frozen threshold on the test set (`scripts/08_diagnostics.py`). This is an audit only: nothing below was used to change the model. Slices were chosen to be interpretable, large enough and operationally relevant. For each one we report the number of readmissions, the prevalence, recall with a 95% Wilson interval, and AP relative to the slice's own prevalence.

Slice (test set, $t = 0.175$)	Readmissions	Prevalence	Recall [95% CI]	AP / prevalence
Overall	2,263	11.4%	0.34	2.1×
No inpatient visit in the past year	1,127	8.6%	**0.15** [0.13–0.17]	2.1×
Discharged home	1,121	9.4%	0.22 [0.19–0.24]	2.1×
Age 80+	491	12.9%	0.27 [0.24–0.31]	1.7×
Two or more inpatient visits	616	21.5%	0.72 [0.68–0.75]	1.5×
Care facility or other hospital	645	16.5%	0.52 [0.48–0.56]	1.8×

Table 8: Slice diagnostics. All slices are evaluated with the same model, threshold and period; full table in `reports/tables/diagnostics_slices.csv`.

The weakest slice is large and important: **half of all early readmissions (1,127 of 2,263) occur in patients without an inpatient visit in the past year, and only 15% of them are flagged**. The interval [0.13, 0.17] excludes the overall recall, so this is not noise. Within this slice the

ranking still works (AP is 2.1 times its prevalence). The failure comes from the global threshold: lower prevalence means lower scores, so few patients cross 0.175.

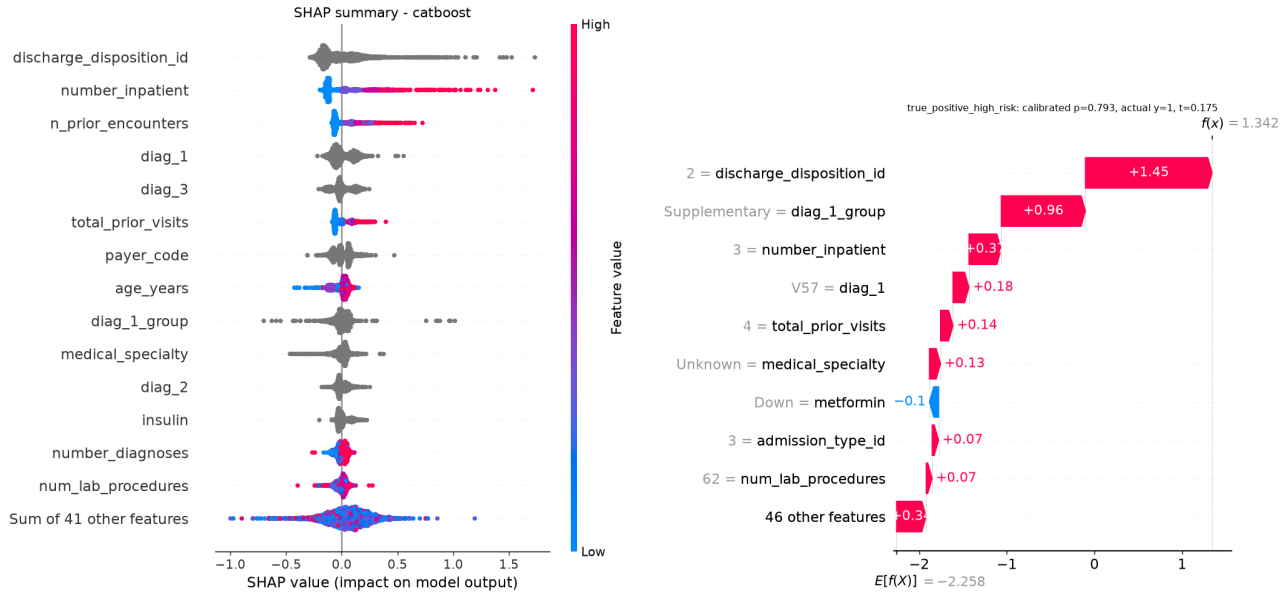
Error analysis. Comparing the 1,502 missed readmissions (FN) with the 761 caught ones (TP):

- 64% vs 22% had no inpatient visit in the past year;
- 66% vs 31% had no earlier stay in the records;
- 58% vs 32% were discharged home;
- 48% of the missed patients received a risk below 0.10.

SHAP confirms it. Among actual readmissions, prior inpatient visits contribute +0.32 log-odds on average for TPs and −0.02 for FNs; discharge destination contributes +0.27 vs −0.02. *Audit finding:* the model relies on utilisation history and treats first-time patients sent home as low risk. *Hypothesis to test:* the information that would reveal their risk (social support, follow-up arrangements) is not recorded. *Next checks:* the same rates among non-readmitted patients, and the same analysis on later data. *Possible actions:* collect post-discharge signals, monitor this slice, or set a slice-specific operating point. No causal claim is made.

10 Explainability

We computed SHAP values with `TreeExplainer` on 3,000 test patients (log-odds scale, before any calibration; one-hot columns summed back to their feature for the other model families). Global importance (mean |SHAP|): discharge disposition (0.154), inpatient visits in the past year (0.150), prior encounters (0.083), primary and tertiary ICD-9 codes (0.075, 0.064), total prior visits (0.062), payer code (0.061), age (0.048), primary diagnosis group (0.046) and medical specialty (0.039).



(a) Global summary (beeswarm). **(b)** Highest-risk readmitted patient ($p = 0.79$).

Figure 4: SHAP explanations of the final CatBoost model on test patients.

Business sense. The ranking matches clinical knowledge. Patients discharged to another facility (skilled nursing, rehabilitation, another hospital) are sicker and less stable; repeated past admissions ("frequent flyers") are the best-documented readmission predictor. Age matters less than expected: its partial dependence is almost flat (Section 10). The diagnosis codes add case-mix information. Four local explanations are saved (`reports/figures/shap_local_*.png`). For the highest-risk readmitted patient, disposition (+1.45 log-odds) and primary diagnosis group (+0.96) dominate. The highest-scoring false alert shows the same pattern: the model cannot see what happens after discharge, which caps its precision. The missed readmission ($p = 0.02$) had low prior utilisation and a low-risk discharge

disposition: nothing in the record announced the return. **Point of attention:** `payer_code` ranks seventh. Insurance can proxy socio-economic status and, indirectly, race, which motivates the audit below.

Three definitions of importance. Session 8 stresses that importance has no single definition, and our results show why. On the tuned LightGBM, *inpatient visits in the past year* ranks 13th by split count but 2nd by total gain: it is used in few splits, but each one improves the objective a lot. Permutation importance on held-out data (final CatBoost, 6,000 test rows, 3 repeats, AP drop) ranks discharge destination (0.053) and inpatient visits (0.032) at the top. Race, which is not an input, has a drop of exactly 0: a sanity check of the method.

Correlated features illustrate the course's warning: inpatient visits and prior encounters have Spearman $\rho = 0.74$. Shuffling them one at a time costs 0.032 and 0.009 AP, but shuffling the four history features together costs 0.057, because each feature substitutes for the others.

Partial dependence and ICE. Setting every patient's number of inpatient visits to the same value and averaging the predictions gives a risk rising from 0.11 (no visit) to 0.30 (seven or more). Length of stay and age are almost flat (0.10–0.12). The ICE curves are roughly parallel, so the average hides no opposite individual responses. Caveat: forcing eight visits on a patient with no prior encounter creates combinations unseen in training. None of these tools is causal: they describe the model, not why patients return.

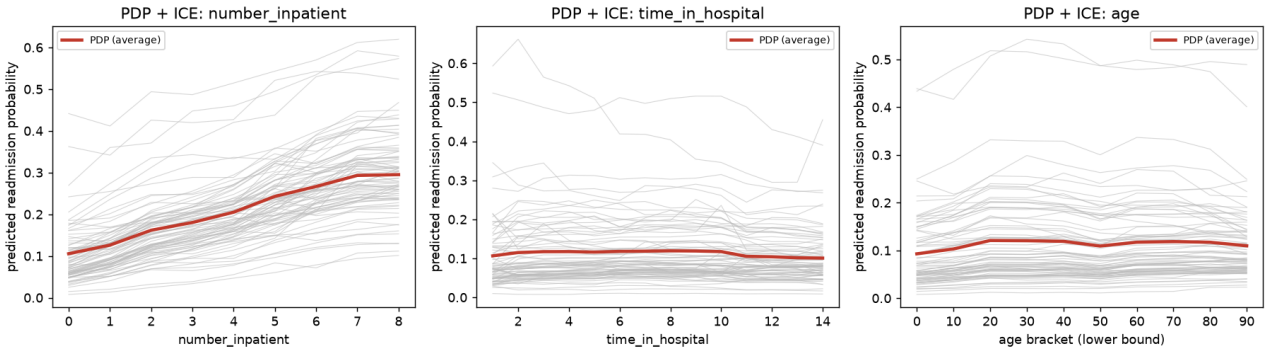


Figure 5: PDP (red) and 80 ICE curves (grey) of the final model on 300 test patients.

11 Fairness and bias audit

Definition. We audit race (`race`, never used as an input) with **equal opportunity**: among patients who will be readmitted, the probability of being flagged (the true-positive rate, TPR) should be the same in every group. A flag gives access to follow-up care and a planned bed, so a lower TPR means a group's at-risk patients are more often left without it. Demographic parity (equal flag rates) is reported but not targeted, because base rates genuinely differ (8.0% for *Unknown* race, 10.2–11.7% elsewhere). Asian and Other (442 test patients) are merged to keep groups estimable.

Mitigation. We use post-processing with group-specific thresholds, the deterministic form of fairlearn's `ThresholdOptimizer` with a TPR-parity constraint. For each group, the threshold is set on the out-of-fold (CV-validation) predictions of the development set so that its TPR equals the overall CV-validation TPR at $t = 0.175$. Thresholds range from 0.127 (*Unknown*) to 0.189 (*Hispanic*). The test set is used only for the before/after comparison.

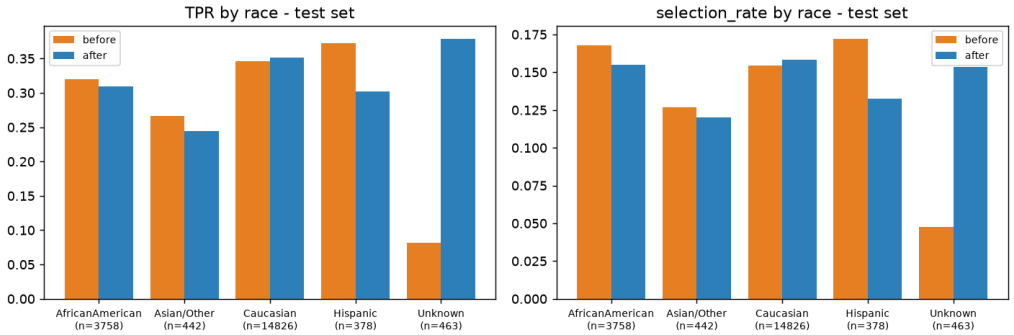


Figure 6: TPR and selection rate by race on the test set, before (global threshold) and after (group thresholds).

Test set	Before (global t)	After (group thresholds)
Equal opportunity difference (TPR max − min)	0.291	**0.134**
Equalized odds difference	0.291	0.134
Demographic parity difference	0.124	0.039
Overall recall / precision	0.336 / 0.249	0.342 / 0.249
Expected cost per discharge	0.494	0.492

Table 9: Fairness before and after mitigation (fairlearn `MetricFrame`).

The mitigation halves the TPR gap at no cost in precision or overall cost. A critical reading is needed, however. The gap is driven by the *Unknown* group, whose TPR rises from 0.08 to 0.38 but which has only 37 readmissions in the test set. Among named groups the gap stays around 0.10 before and after, within the sampling noise of groups with 40–400 positives (standard error ≈0.08 for a TPR estimated on 40 positives). Using race at decision time is also a disparate-treatment issue in many jurisdictions. In practice we would keep race out of the decision, monitor TPR by group over time, and investigate why race is missing for some patients (a data-quality signal in its own right). Gender shows no gap (TPR 0.33 vs 0.35); the youngest and oldest age brackets are too small to conclude.

12 Limitations and conclusion

Limitations.

- Performance is modest (AP 0.24, ROC-AUC 0.68): the main drivers of a readmission occur after discharge and are not recorded.
- The data are 1999–2008 with no dates, so no temporal validation was possible.
- Costs are assumptions, although the decision rule makes them explicit and the cost curve is flat around the optimum.
- CatBoost was only lightly tuned, and the target encoder's internal folds are not grouped by patient. That affects only the training-time encoding, never validation.
- Small racial groups make fairness estimates noisy.

Conclusion. We built an end-to-end, leakage-safe supervised-learning system: patient-disjoint validation, all preprocessing fitted per fold, a single test evaluation, Optuna tuning tracked in MLflow, calibrated probabilities, a cost-based decision rule, SHAP explanations, a fairness audit, and a FastAPI endpoint that serves the frozen pipeline.

Model family and the encoding of categoricals bought performance; resampling, class weights and long tuning did not. The final CatBoost model ranks moderately well, but its probabilities are reliable (2,258 expected vs 2,263 observed readmissions). For a bed-planning use case this matters more than a few points of AUC.

Use of AI assistants

Claude (Anthropic) was used as a coding and drafting assistant to help structure the Python package and scripts, support implementation of the experiment pipeline, and draft parts of the slides and report. ChatGPT (OpenAI) was also used to support debugging, conceptual review, interpretation of modelling choices and metrics, and preparation of the presentation.

The exploratory data analysis was carried out by the group, and the course material was studied independently in order to challenge AI-generated coding choices, verify that the modelling decisions were consistent with the concepts taught in class, and check the coherence of the project as a whole. All modelling decisions, validation choices, results and interpretations were reviewed by the group, and every member can explain and defend the final methodology and conclusions.

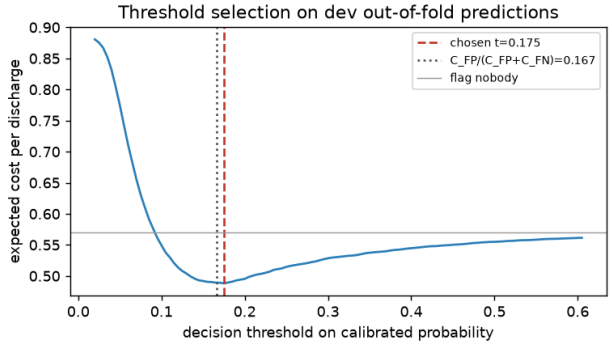
A Reproducibility

- Environment: Python 3.13, pinned `requirements.txt`, `pip install -e .`
- Order of execution: `01_eda`, `02_experiments`, `03_tune`, `04_finalize`, `05_explain`, `06_fairness`, `07_export_runs`, `08_diagnostics`, `09_leakage_checks`.
- Configuration: every choice lives in `configs/config.yaml`, including the cohort exclusions, the CV, the costs, the Optuna budgets and MLflow.
- Determinism: one seed (42) for all splits, models and samplers.
- Tests: `pytest` (11 tests) covers ICD-9 grouping, the past-only history feature, rare-category pooling learned on fit data only, monotone calibrators, ECE and the API end to end.
- Serving: `uvicorn api.app:app`, then `POST /predict` with the encounter's fields; missing fields take the documented defaults.

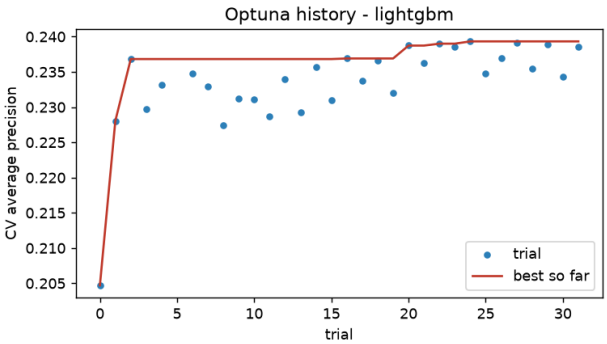
B Tuned hyperparameters

Model	Best configuration (mean CV-validation AP)
Logistic regression	L1, $C = 0.213$, no class weight (0.219)
Random Forest	200 trees, max depth 25, min samples leaf 29, max features 0.19, max samples 0.37, no class weight (0.227)
LightGBM	900 trees, learning rate 0.006, 33 leaves, max depth 6, min child samples 27, subsample 0.91, colsample 0.38, $\alpha = 0.18$, $\lambda = 0.016$, scale_pos_weight 1.41 (0.239)
CatBoost	900 iterations, learning rate 0.067, depth 7, ℓ_2 leaf reg 1.06, no class weight (0.248)

C Additional figures



(a) Threshold selection on the out-of-fold (CV-validation) predictions.



(b) Optuna history, LightGBM.

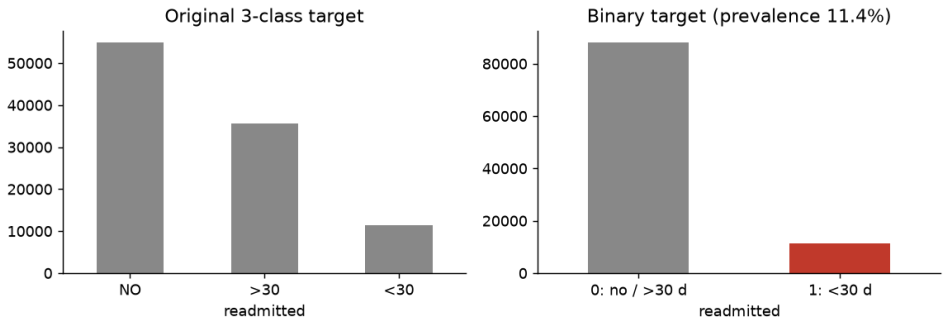


Figure 8: Original three-class target and binary recoding.